

MASt3R-SLAM: Real-Time Dense SLAM with 3D Reconstruction Priors

Nathan Rouillé
Course: NPM3D
[GitHub Repository](#)

March 2026

1 Introduction

1.1 General Context of Monocular SLAM

Visual simultaneous localisation and mapping (SLAM) is a foundational building block for today’s robotics and augmented reality products. Historically, to obtain accurate pose estimation, sparse monocular SLAM focuses on jointly solving for camera poses and a select number of 3D landmarks. While it can be very accurate given sufficient features, it lacks a dense scene model, which is useful for both robust tracking and more explicit reasoning over geometry. Performing dense SLAM from only 2D images requires reasoning over time-varying poses and camera models, as well as 3D scene geometry.

1.2 The Paradigm Shift : Pre-trained 3D Priors

To solve this inverse problem of large dimensionality, a variety of priors have been proposed. Recently, a paradigm shift in structure-from-motion (SfM) was introduced thanks to the two-view 3D reconstruction priors, created by DUS3R and its successor MAST3R. These networks output pointmaps directly from two images in a common coordinate frame. They implicitly solve the complex sub-problems of correspondence, poses, camera models, and dense geometry in a joint framework. However, while these priors are powerful in an offline setting, a SLAM system receives data incrementally and must maintain real-time operation.

1.3 The Contribution of MAST3R-SLAM

The paper *MASt3R-SLAM: Real-Time Dense SLAM with 3D Reconstruction Priors* proposes the first real-time monocular dense SLAM system based on the MAST3R 3D reconstruction prior. The system is capable of producing globally consistent poses and dense geometry while operating at 15 FPS. A major quality

of this approach is that it makes no assumption on a fixed or parametric camera model except a unique camera centre that all rays pass through. To achieve this performance, the authors introduce efficient methods for pointmap matching, camera tracking, local fusion, graph construction, loop closure, and second-order global optimisation.

1.4 Objective and Structure of the Report

This report aims to analyze the architecture of MAST3R-SLAM while confronting theory with practice. To offer a fluid reading experience, we will adopt an interleaved approach: for each key concept presented by the article, we will detail the underlying theory before validating it (or showing its limitations) through our own experiments.

Specifically, this report will explore four major experimental axes:

- The algorithmic efficiency of **iterative projective matching** against classical approaches.
- The crucial importance of the **directional error formulation (Ray Error)** for camera tracking to be robust to depth inaccuracies.
- The impact of different **local pointmap fusion** strategies on the consistency of the geometric map.
- The role of **Loop Closure** in mitigating drift over long trajectories.

By combining the theoretical analysis of the article and these practical implementations, this report will highlight the strengths of this new paradigm while identifying its current limits.

2 System Architecture and Experimental Validations

This section details the fundamental components of MAST3R-SLAM. For each module, we expose the theoretical approach adopted by the authors before confronting it with our own ablation experiments (every result is available in the results directory in the github repository), in order to measure the real impact of these design choices.

It is important to define our general experimental protocol. To ensure deterministic and reproducible results, all evaluations were run in the paper’s default evaluation mode (`single_thread: True` and temporal `subsample: 2`), which trades real-time 15 FPS performance for strict reproducibility. Furthermore, because monocular SLAM is inherently ambiguous in scale, all our Absolute Trajectory Error (ATE) measurements are computed using a complete Umeyama alignment (`align=True, correct_scale=True`). This scale correction ensures that the ATE is not biased by absolute scale drift and remains a fair comparative metric across all experiments.

2.1 Pointmap Matching

2.1.1 Theory : Avoiding the Bottleneck

Correspondence is a fundamental component of SLAM that is required for both tracking and mapping. Given the pointmaps and dense features generated by MAST3R, a naive brute-force matching has quadratic complexity, as it is a global search over all possible pairs of pixels. The classical alternative of a k-d tree over 3D points, used by DUST3R, will find many inaccurate matches if there are errors in the pointmap predictions and it is poorly parallelisable. MAST3R proposes a coarse-to-fine scheme to handle the global search, but the runtime is on the order of seconds for dense pixel matching.

To tackle this problem, MAST3R-SLAM introduces an iterative projective data association inspired by local optimisation search. Lacking prior knowledge of the parametric camera model, the system normalises the reference pointmap X_i^i to construct a generic camera model defined by unit rays : $\psi(X_i^i)$. Each 3D point $x \in X_i^j$ is then projected independently by iteratively optimising its pixel coordinates p in the reference frame to minimise the ray error relative to the target ray:

$$p^* = \arg \min_p \|\psi([X_i^i]_p) - \psi(x)\|^2$$

Minimizing this Euclidean distance between normalised vectors is equivalent to minimizing the angle θ between the two rays:

$$\|\psi_1 - \psi_2\|^2 = 2(1 - \cos \theta)$$

This geometric projection provides a good initialisation, then the system conducts a final coarse-to-fine search by updating the pixel location to the most similar MAST3R feature in a local patch window. This two-step process, implemented in custom CUDA kernels, is massively parallelisable and converges for almost all valid pixels within 10 iterations, dropping the matching time to around 2 milliseconds.

2.1.2 Experiment and Analysis

To evaluate the importance of this approach, we replaced the paper’s projective matching with two alternatives. The implementation required modifying `mast3r_slam/matching.py` by turning the `match()` function into a dispatcher driven by `config/base.yaml`. We added `match_kdtree()` (relying on `scipy.spatial.KDTree` for 3D coordinate matching) and `match_bruteforce()` (normalizing descriptors and computing block-wise cosine similarity to enforce mutual reciprocity). The runner `experiments/run_matching_ablation.py` was used to record execution times and the Absolute Trajectory Error (ATE). The ATE, specifically computed here as a Root Mean Square Error (RMSE), evaluates the global consistency of the estimated camera poses by measuring the absolute euclidean distance between the predicted trajectory and the ground truth. Because it robustly captures the overall accuracy of a SLAM system, it

serves as our primary quantitative metric throughout all the experiments presented in this report.

Matching Method	fr1/desk		fr1/room	
	ATE (m)	Time (s)	ATE (m)	Time (s)
Iterative Projection	0.033	165.0	0.118	337.9
k-d tree	0.042	417.5	0.091	1077.0
Brute-force	0.820	5530.0	OOM	OOM

Table 1: Comparison of ATE (RMSE) and total execution time across matching methods.

Analysis of the results: As summarized in Table 1, our experiments clearly highlight both the performance and hardware bottlenecks of naive matching approaches. It is important to note that we executed these runs on a cluster equipped with an NVIDIA Tesla P100 GPU limited to 16 GB of VRAM. This hardware constraint, compared to the NVIDIA RTX 4090 (24 GB VRAM) utilized by the authors, heavily impacts the system and explains the massive gap in overall execution times relative to the real-time capabilities reported in the paper.

The brute-force run highlighted a major hardware obstacle not explicitly detailed in the article: the VRAM bottleneck. With a full-resolution image ($N = 196608$ pixels) and descriptors of dimension $d = 24$, MAST3R quickly saturates the graphics memory. To make the brute-force run viable, we were forced to implement a spatial `bruteforce_subsample = 8` (dropping to ~ 24576 pixels) and block-wise calculations (`bruteforce_chunk = 256`). This chunk size was not a performance choice, but the strict maximum allowed to fit the 200 MB similarity matrix into the ~ 1 GiB of VRAM remaining. While these empirical compromises allowed us to process the short *fr1/desk* sequence in over 1.5 hours (5530 s), the severe downsampling destroyed the matching accuracy (ATE of 0.820 m). Furthermore, the method still failed entirely on *fr1/room*. Because *fr1/room* is a longer sequence generating approximately 500 keyframes, the graph accumulates a massive number of edges. Concatenating these indices in GPU memory within the `prep_two_way_edges` function triggered a fatal OOM error, as it exceeded the ~ 1.28 GiB of VRAM left available after loading the network weights. This experiment empirically demonstrates that the iterative projective matching is not merely a speed optimisation choice, but an absolute necessity to maintain the memory footprint of a dense SLAM system within viable hardware limits.

2.2 Camera Tracking

2.2.1 Theory : The Robustness of Directional Error

A key component of SLAM is low-latency tracking to estimate the relative transformation T_{kf} between the current frame and the last keyframe. The straightforward approach would consist of minimizing the 3D Euclidean error (point error) between the matched points. However, the authors emphasize that this 3D point error is easily skewed by errors in the pointmap predictions, as inconsistent predictions in depth are relatively frequent.

To solve this, they exploit the central camera assumption to calculate a directional ray error instead. By normalising both points to unit norm using the function $\psi(\cdot)$, the system minimises the angular errors between canonical rays and corresponding predicted rays. This gives the objective function, the ray error E_r :

$$E_r = \sum_{m,n \in m_{f,k}} \left\| \frac{\psi(\tilde{X}_{k,n}^k) - \psi(T_{kf} X_{f,m}^f)}{w(q_{m,n}, \sigma_r^2)} \right\|_\rho$$

where $\|\cdot\|_\rho$ represents the Huber norm and $w(\cdot)$ applies a per-match confidence weighting from MAST3R’s outputs. Because angular errors are bounded, ray-based errors are highly robust against outliers and incorrect depth predictions. A small distance error term is also included to prevent the system from becoming degenerate under pure rotation. Finally, the system efficiently solves for updates to the relative pose $\tau \in \text{sim}(3)$ using Gauss-Newton in an iteratively reweighted least-squares (IRLS) framework, updating the pose via $T_{kf} \leftarrow \tau \oplus T_{kf}$.

While the system inherently operates without a known camera model to calculate the ray error, it can also leverage intrinsic calibration when available to improve the accuracy of both pointmap matching and camera tracking. In such cases, the optimisation changes the residuals to operate in pixel space rather than ray space. It minimizes the classical reprojection error between the 2D pixel and the projection of its matched 3D point using the provided camera model.

2.2.2 Experiment and Analysis

To validate this design choice, we modified `mast3r_slam/tracker.py` to add an `opt_pose_point_sim3()` method, which implements Gauss-Newton optimisation using the direct 3D residual ($r = X_k - T_{C_k C_f} X_f$) and left-perturbation Jacobians on $\text{Sim}(3)$. Furthermore, `mast3r_slam/global_opt.py` was updated to include a PyTorch backend (`solve_GN_points()`) for global optimisation with configurable point downsampling. The ablation was run via `experiments/run_error_ablation.py`.

Analysis of the results: As shown in Table 2, the ray error is undisputedly superior, validating the authors’ choice where ray formulation improves performance over using the 3D point error containing inaccurate depth predictions. Nevertheless, our degradation is much more severe ($\sim 20\times$) than that reported

Formulation	fr1/desk	fr1/room
Ray Error (Paper)	0.038 m	0.118 m
Point Error (Ablation)	0.742 m	0.791 m

Table 2: Comparison of the ATE (RMSE) depending on the optimization error formulation.

in the article ($\sim 1.5\times$). This massive gap is directly explained by three necessary empirical compromises in our Python-based implementation. First, to maintain reasonable execution times, we applied a spatial `point_subsample` = 4 (processing only $\sim 25\%$ of pixels). Second, we maintained `max_iters` = 10, which is sufficient for the paper’s highly optimized CUDA kernels but insufficient for our Python backend’s convergence. Finally, and most importantly, the confidence weighting `sigma_point` = 0.05 was inherited directly from the ray formulation. While 0.05 is appropriate for dimensionless angular residuals (~ 0.003 rad), it is severely miscalibrated for metric Euclidean residuals (~ 0.1 - 0.5 m), fundamentally skewing the information matrix during optimization.

2.3 Pointmap Fusion

2.3.1 Theory : Filtering Geometric Noise

Unlike offline SfM which optimizes all maps simultaneously, real-time SLAM must maintain map coherence efficiently. Tracking against a new frame provides new geometric information, which MAST3R-SLAM leverages by updating a canonical pointmap $\tilde{X}_k^k$ via a running weighted average filter. After solving for the relative pose T_{kf} , the system incrementally updates the pointmap and its accumulated confidence $\tilde{C}_k^k$ using the current frame’s transformed pointmap X_f^k and its associated confidence C_f^k :

$$\tilde{X}_k^k \leftarrow \frac{\tilde{C}_k^k \tilde{X}_k^k + C_f^k (T_{kf} X_f^k)}{\tilde{C}_k^k + C_f^k}, \quad \tilde{C}_k^k \leftarrow \tilde{C}_k^k + C_f^k$$

This process merges information from many viewpoints, filtering not only estimates of the geometry but also the camera model itself (which is defined by the rays). Hence, it yields the benefit of leveraging information from all frames without having to explicitly optimise for all camera poses and store all the predicted pointmaps from the decoder in the backend. The authors experimented with different update methods and concluded that the weighted average was best for maintaining coherence while filtering out noise.

2.3.2 Experiment and Analysis

We automated the evaluation of the four fusion strategies proposed by the authors by dynamically generating temporary configurations inheriting from

`eval_no_calib.yaml` via our `experiments/run_fusion_ablation.py` runner. To make this work, we first had to correct a bug in `mast3r_slam/frame.py` where the `Frame` dataclass failed to initialize the `score` attribute when reconstructing from shared memory, previously causing an `AttributeError` during the median evaluation.

Fusion Mode	fr1/desk	fr1/room
recent	0.094 m	0.168 m
median	0.089 m	0.162 m
first	0.036 m	0.159 m
weighted_pointmap (Default)	0.033 m	0.118 m

Table 3: ATE (RMSE) according to the fusion strategy of the canonical pointmap.

Analysis of the results: Table 3 corroborates the findings of the article: the weighted fusion is the most performant. The analysis of the alternative strategies is particularly instructive. Fixing the pointmap on the first observation is more robust than keeping the last, because it prevents the map from drifting due to degraded MAST3R predictions later in the sequence. The weighted average offers the optimal compromise by stabilizing the global map without discarding valuable new geometric observations.

2.4 Loop Closure & Backend Optimisation

2.4.1 Theory: Correcting the Drift Inherent to Priors

Despite the power of local filtering, MAST3R predictions can still have inconsistencies and correlated errors in 3D geometry, which inevitably cause drift over time. To manage this, the system maintains a pose graph where new nodes (keyframes) are added whenever tracking matches fall below a threshold ω_k . To close both small and large loops, the system adapts the Aggregated Selective Match Kernel (ASMK) framework to retrieve incrementally similar images using encoded MAST3R features. If retrieval scores and subsequent pixel matches exceed the thresholds ω_r and ω_l , preventing false loops and maintaining efficiency, validated candidates generate new bidirectional edges in the backend graph. This same ASMK retrieval mechanism naturally extends to relocalisation: if the system loses tracking due to an insufficient number of matches, the database is queried with a stricter score threshold to confidently find matches and attach the frame to the graph to resume tracking.

Because frontend tracking only filters locally, large-scale drift must be corrected globally. The goal of the backend optimisation is to then achieve global consistency across all poses and geometry by jointly minimizing the ray error for all edges using an efficient second-order optimisation. Specifically, this is solved via Gauss-Newton with sparse Cholesky decomposition, fixing the first 7-DoF Sim(3) pose to handle the gauge freedom.

2.4.2 Experiment and Analysis

In order to isolate the impact of Loop Closure, no source code modification was required. We simply blocked ASMK queries by dynamically setting the configuration hyperparameter `retrieval.k`: 0 in our `experiments/run_loopclosure_ablation.py` runner, evaluating the effect on both short and long sequences across the TUM and 7-Scenes datasets.

Sequence	Dataset	With LC	Without LC	Gain (Δ)
heads (~ 1000 fr)	7-Scenes	0.0149 m	0.0399 m	−62%
fr1/desk (~ 600 fr)	TUM	0.0326 m	0.0383 m	−15%
redkitchen (~ 2500 fr)	7-Scenes	0.0414 m	0.0562 m	−26%
fr1/room (~ 1350 fr)	TUM	0.1183 m	0.2114 m	−44%

Table 4: Impact of Loop Closure activation on ATE (RMSE).

Analysis of the results: Loop Closure improves the trajectory in every cases, justifying the additional computation cost of the backend. Unsurprisingly, the most spectacular absolute gain concerns *fr1/room* (−0.093 m), a closed trajectory navigating through an entire room. Interestingly, the massive relative improvement on *heads* is explained by the fact that it is a very low-texture scene (a close-up of a face). In such conditions, ray error tracking drifts quickly even over a few frames, making loop closure indispensable to correct these small local drifts. It should be noted that due to disk quota constraints on our cluster, our 7-Scenes evaluation was limited to a single sequence (`seq-01`) per scene, whereas the original paper likely averages results across multiple sequences. This explains why our results exhibit higher variance, and why the gain on *red-kitchen* (−26%) is lower than expected for a "long" sequence, as `seq-01` alone may not constitute a complete loop closure circuit.

3 Critique and Limitations

While MAST3R-SLAM demonstrates impressive capabilities by bridging foundation models and real-time SLAM, both the original authors and our experiments highlight some limitations.

From a theoretical standpoint, the authors acknowledge that the system relies heavily on the MAST3R decoder, which operates at full resolution and consumes roughly 64% of the total tracking time. This computational bottleneck currently prevents the system from being deployed on resource-constrained edge devices.

Furthermore, while the uncalibrated ray formulation gracefully handles unknown camera models, the underlying MAST3R prior was trained on pinhole images. Consequently, the raw geometric predictions can degrade when confronted with extreme, out-of-distribution camera distortions not seen during

training. To empirically test this limitation, we captured custom video sequences using a wide-angle camera. Interestingly, the system proved resilient to these moderate distortions and maintained robust tracking, suggesting that truly extreme lenses (heavy fisheyes) are required to expose this geometric flaw. However, our custom recordings highlighted another problem: the system is sensitive to motion blur. In highly dynamic sequences, blurry frames severely disrupted MAST3R’s feature extraction and pointmap predictions, leading to significant degradation and failures in the final 3D reconstruction.

Beyond the theoretical constraints, our empirical results showed operational and engineering limitations:

- **Hardware and Memory Constraints:** The reliance on dense, high-dimensional descriptors creates an extreme VRAM bottleneck. Our experiments showed that the MAST3R network alone occupies ~ 14 GiB of VRAM. As evidenced in our matching ablation, without the highly optimized iterative projective method, naive approaches immediately trigger Out-Of-Memory (OOM) errors even on our 16 GB Tesla P100 when accumulating graph edges or computing similarity matrices.
- **Engineering Complexity and Sensitivity:** Our Ray vs. Point error experiment revealed that substituting the paper’s custom CUDA kernels with a standard PyTorch backend introduces severe convergence issues. Furthermore, the system is highly sensitive to hyperparameter tuning: we demonstrated that applying the paper’s default dimensionless confidence weighting ($\sigma = 0.05$) to metric 3D point residuals dramatically skewed the optimization, causing a huge degradation in accuracy.
- **Evaluation Constraints:** Due to disk quota limits on our computing cluster, our 7-Scenes evaluations were restricted to a single sequence (seq-01) per scene. Unlike the original paper which averages results across multiple trajectories, our approach introduces higher variance and explains why some long-term loop closure gains were less pronounced than expected.

4 Conclusion

The paper *MASt3R-SLAM* presents a paradigm-shifting approach by building a real-time dense SLAM system around a two-view geometric prior. By utilizing a generic central camera model defined by rays, it successfully handles in-the-wild videos and achieves state-of-the-art performance without requiring known parametric calibration, while remaining flexible enough to leverage calibration when available.

Through our reproduction and experiments, we empirically validated the core design choices of the system. We demonstrated that iterative projective matching is an absolute necessity to bypass quadratic complexity and severe VRAM limitations. We confirmed that the directional ray error formulation

is crucial to absorb the depth inconsistencies inherent to network predictions. Furthermore, our results showed that a weighted canonical pointmap fusion optimally balances map stability with new observations, and that loop closure via ASMK retrieval effectively mitigates the long-term drift typical of 3D priors.

Ultimately, our analysis of MAST3R-SLAM proves that foundation models for 3D reconstruction can be effectively used for real-time spatial intelligence. However, it also serves as a reminder that deploying these heavy theoretical priors in real-time scenarios requires rigorous, highly optimized geometric backends and careful systems engineering to overcome profound hardware limitations.